

FASE CLIENTE — SPRINT 3.-

— MIGRACIÓN DEL ESCAPARATE VIRTUAL A REACT

Entrega Final: fecha asignada a finales de mayo.

CONTEXTO DEL PROYECTO

Tras completar los sprints anteriores, el escaparate virtual ya dispone de varias funcionalidades implementadas en JavaScript Vanilla:

- formularios interactivos con validación,
- carrito de la compra,
- buscador de productos mediante AJAX,
- galería dinámica de productos.

El cliente está satisfecho con el funcionamiento actual, pero desea **modernizar la arquitectura del frontend** para facilitar el mantenimiento del código, mejorar la reutilización de componentes y preparar el proyecto para su futura integración con un backend real.

Por este motivo, el cliente solicita **adaptar el proyecto actual para utilizar React**, manteniendo las funcionalidades desarrolladas hasta ahora.

El objetivo de este sprint es **reestructurar la aplicación utilizando un enfoque basado en componentes**, conservando el comportamiento existente.

HISTORIAS DE USUARIO

HU-CL-S3-01: MODERNIZACIÓN TECNOLÓGICA

Como cliente,
quiero que el escaparate virtual esté desarrollado utilizando React,
para que la aplicación sea más modular, escalable y fácil de mantener.

HU-CL-S3-02: REUTILIZACIÓN DE COMPONENTES

Como desarrollador,
quiero organizar la interfaz en componentes reutilizables,
para facilitar la evolución futura de la aplicación.

Estas historias de usuario deberán documentarse en el repositorio siguiendo el formato utilizado en los sprints anteriores.

OBJETIVO DEL SPRINT

Migrar la aplicación actual a **React**, reorganizando el código en componentes y manteniendo las funcionalidades implementadas en los sprints anteriores.

REQUISITOS FUNCIONALES

La aplicación React debe mantener las funcionalidades desarrolladas previamente:

1. ESCAPARATE DE PRODUCTOS

- Mostrar una **galería de productos o servicios**.
- Los productos deben obtenerse mediante **peticiones AJAX** a un API.

2. BUSCADOR DE PRODUCTOS

- Debe existir un **campo de búsqueda**.
- La búsqueda debe filtrar productos por:
 - nombre
 - descripción.
- Los resultados deben actualizarse **dinámicamente**.

3. CARRITO DE LA COMPRA

El carrito debe permitir:

- añadir productos,
- indicar cantidad,
- visualizar número de elementos en el icono del carrito,
- mostrar el contenido del carrito,
- eliminar productos,
- calcular el precio total.

4. FORMULARIO DE USUARIO

El formulario de registro implementado en el Sprint 1 deberá seguir funcionando con:

- validaciones personalizadas en JavaScript,
- mensajes de error interactivos,
- comportamiento sin recarga de página.

REQUISITOS TÉCNICOS

1. El proyecto debe desarrollarse utilizando **React**.
2. La aplicación debe organizarse mediante **componentes**, como por ejemplo:
 - Header
 - ProductGallery
 - ProductCard
 - SearchBar
 - Cart
 - CartItem
 - RegisterForm
3. Se debe utilizar **gestión de estado** para manejar:
 - productos cargados desde el API,
 - resultados de búsqueda,
 - contenido del carrito.
4. Las peticiones al API deben realizarse mediante:
 - `fetch`
o
 - `axios`.
5. La aplicación debe mantener una **estructura clara de componentes y carpetas**, por ejemplo:

```
src/  
├── components/  
├── pages/  
├── services/  
├── hooks/  
└── App.jsx
```
6. No es necesario utilizar librerías avanzadas de gestión de estado (Redux, Zustand, etc.).
Se recomienda utilizar **useState** y **useEffect**.

INTEGRACIÓN CON EL PROYECTO

- El comportamiento de la aplicación debe mantenerse respecto a los sprints anteriores.
- El objetivo de este sprint **no es añadir nuevas funcionalidades**, sino **adaptar la arquitectura del frontend a React**.
- El diseño visual existente tiene que seguir las especificaciones de todos los módulos.

3. TESTS FUNCIONALES — APLICACIÓN REACT

Se deben entregar:

- Grabaciones, scripts o exportaciones de las pruebas realizadas con **Selenium**, **Katalon Recorder** u otra herramienta similar.

PRUEBAS MÍNIMAS NECESARIAS

Las pruebas funcionales deberán cubrir, como mínimo:

- carga inicial de productos desde el API,
- funcionamiento del buscador,
- adición de productos al carrito,
- eliminación de productos del carrito,
- cálculo correcto del total del carrito,
- funcionamiento del formulario de registro.

Los tests deberán almacenarse en **su propia carpeta dentro del repositorio**.

MATERIAL A ENTREGAR

En una **nueva rama del repositorio de GitHub**, se deberá subir:

PROYECTO REACT

El proyecto deberá incluir:

- código fuente completo de la aplicación,
- estructura organizada de componentes,
- servicios de acceso al API.

DOCUMENTACIÓN

Un fichero **README.md** que incluya:

- explicación de la arquitectura del proyecto React,
- descripción de los componentes principales,
- explicación de la gestión de estado,
- instrucciones para ejecutar el proyecto (`npm install`, `npm run dev`, etc.).

PUBLICACIÓN

La aplicación deberá publicarse utilizando **GitHub Pages**.